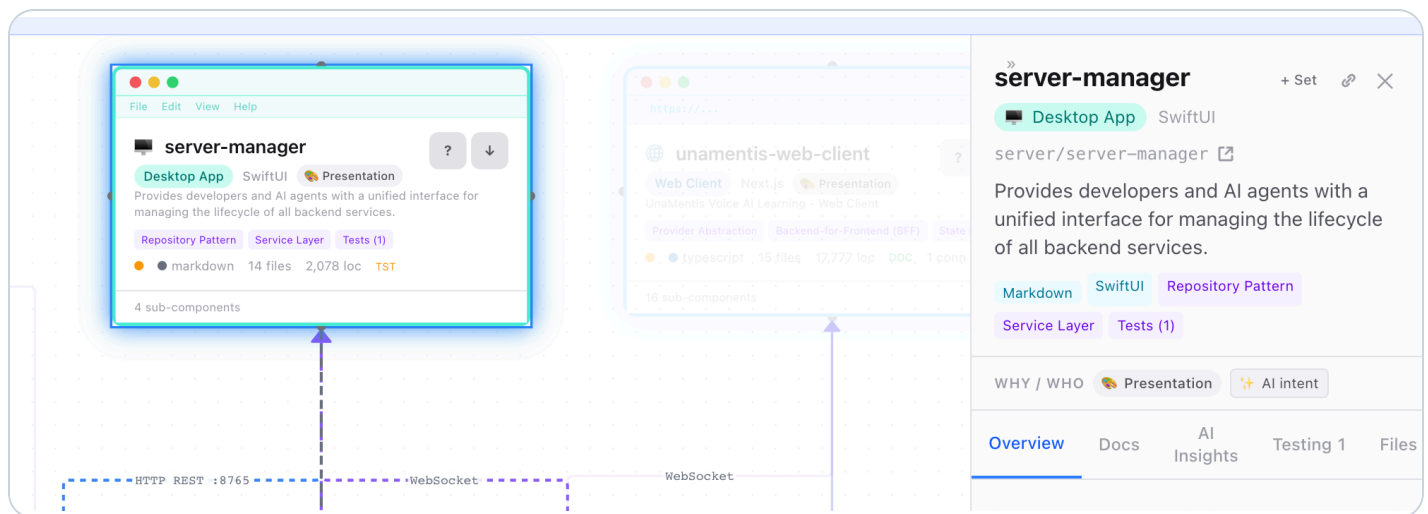


Living maps of real software, deep enough to reach the code.

SysCorpus turns a codebase into an interactive, evidence-aware map and keeps that map current as the code changes. People and AI agents can inspect the same system from the whole architecture through its dependencies and down to files, symbols, and source.



A direct, focused browser capture from a real development codebase: a selected component, nearby relationships, and its evidence-bearing detail panel. This is product evidence, not a demo announcement. Launch captures will replace it only after a public SysCorpus map has graduated.

8

question-specific lenses over one map

12

read-only tools for AI agents

9

full parser families, plus detection tiers

1

CycloneDX SBOM beside each dependency-bearing repository map



Richard Amerman · Creator

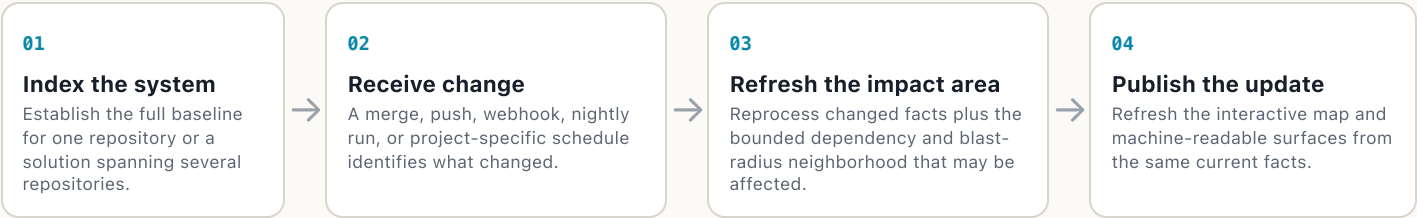
SysCorpus is currently an independent project, created and led by Richard Amerman. The intent is to form a company around the work as it matures.

syscorpus.com
info@syscorpus.com

From source code to a map people can reason with.

The architecture is not maintained as a separate drawing and is not normally rebuilt wholesale. A persistent fact store accepts changes and refreshes the affected system neighborhood.

AI can create working software faster than a team can form a reliable mental model of it. SysCorpus addresses that comprehension gap with a navigable representation of the system, grounded in the system itself.



CONTINUOUS LOOP The cadence belongs to the project: after every accepted change at one extreme, nightly or scheduled at another. A full re-index remains available for first setup, recovery, or an analysis-rule change, but it is not the intended day-to-day maintenance path.

**Change-aware extraction**
Source is parsed without requiring an AI model. Facts are kept in SQLite and keyed by content hash, so a maintenance run can update changed files without parsing unchanged files again.

**Derived system model**
Components and relationships are assembled into a hierarchy, then projected into a single JSON file or a lightweight manifest with lazy detail shards.

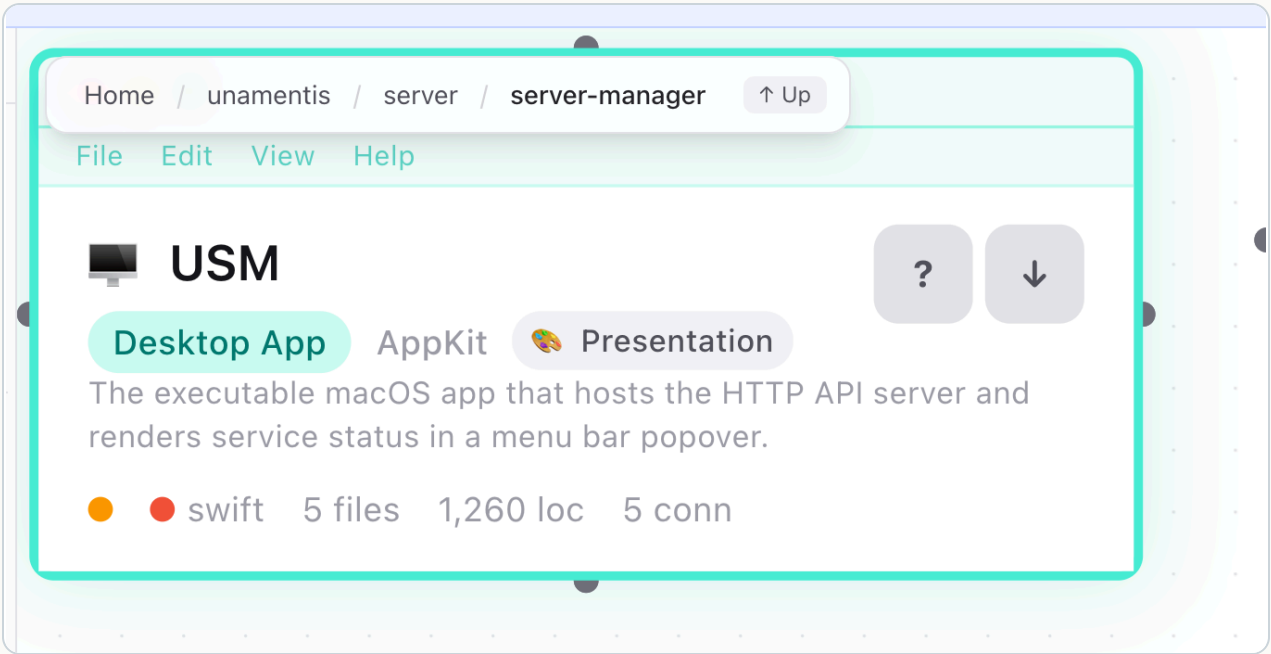
**Optional interpretation**
AI enrichment is additive and off by default. When enabled, claims must cite repository or store evidence. Unsupported answers are narrowed or declared as honest gaps.

LAYER	WHAT IT CAN REPRESENT	EXAMPLES
Structure	Applications, services, packages, modules, screens, files, symbols, and nested component trees.	web client API server class function
Connections	Imports, HTTP and port references, Docker links, gRPC, WebSocket, database, navigation, and related edges.	runtime flow dependency source link
Operational facts	Declared and locked dependencies, direct/transitive scope, versions, pin state, language and SDK targets, vendored code, ports, endpoints, environment variables, tests, and CI status.	CycloneDX SBOM supply chain live status
Behavior and meaning	User flows, capabilities, data entities, business rules, architecture findings, documentation content, and change activity.	flow rules blast radius

Language and supply-chain reach today. Full parsing is implemented for Swift, Python, Rust, TypeScript/JavaScript, Go, Ruby, Java, C#, and C/C++. Seven more language families are detected and measured. Manifest and lockfile readers cover npm, PyPI, Cargo, Go Modules, RubyGems, NuGet, SwiftPM, and CocoaPods; each record retains evidence and parse failures remain visible warnings.

Start with the whole. Follow the question. End at the code.

The interface is layered for readers who need orientation first and technical evidence later. It does not assume they already know the stack.



Direct browser capture after drilling into one subsystem. The complete component remains readable at print scale, and the breadcrumb preserves the route back to the whole system. The cover shows the complementary component-detail view. Final collateral should replace both with one continuous screenshot from the first public SysCorpus map.

Structure How is the system composed?	Inventory Which dependencies, SDK targets, ports, and external services are present?	Flow How does a person move through the application?	Activity Where is code changing, who knows it, and where are the hotspots?
Capability What can the system do, beyond its file structure?	Data What entities exist and how do they relate?	Rules What business rules are encoded, and where?	Design Where is structure under stress, and what could break?

A lens only appears when the dataset can answer its question. Empty navigation is not presented as capability. In the Design lens, blast radius highlights what depends on a selected component, what that component depends on, and the unaffected remainder.

Universal search Components, files, symbols, and the contents of Markdown documentation.	Deep links Share viewer state or arrive by file and line, directly at the owning symbol.	Review loop Annotate a component, file, or symbol and export structured implementation prompts.	Guided tours Walk a code-anchored sequence with narration, evidence links, and staleness checks.	Supply-chain surface Dependencies, versions, pin state, direct/transitive scope, SDK targets, evidence, and the full SBOM.
--	--	---	--	--

Recognizable before it is technical

Primary components use frames that match their role: phone, browser, server rack, watch, desktop window, terminal, or service. Runtime connections and structural dependencies are visually distinct.

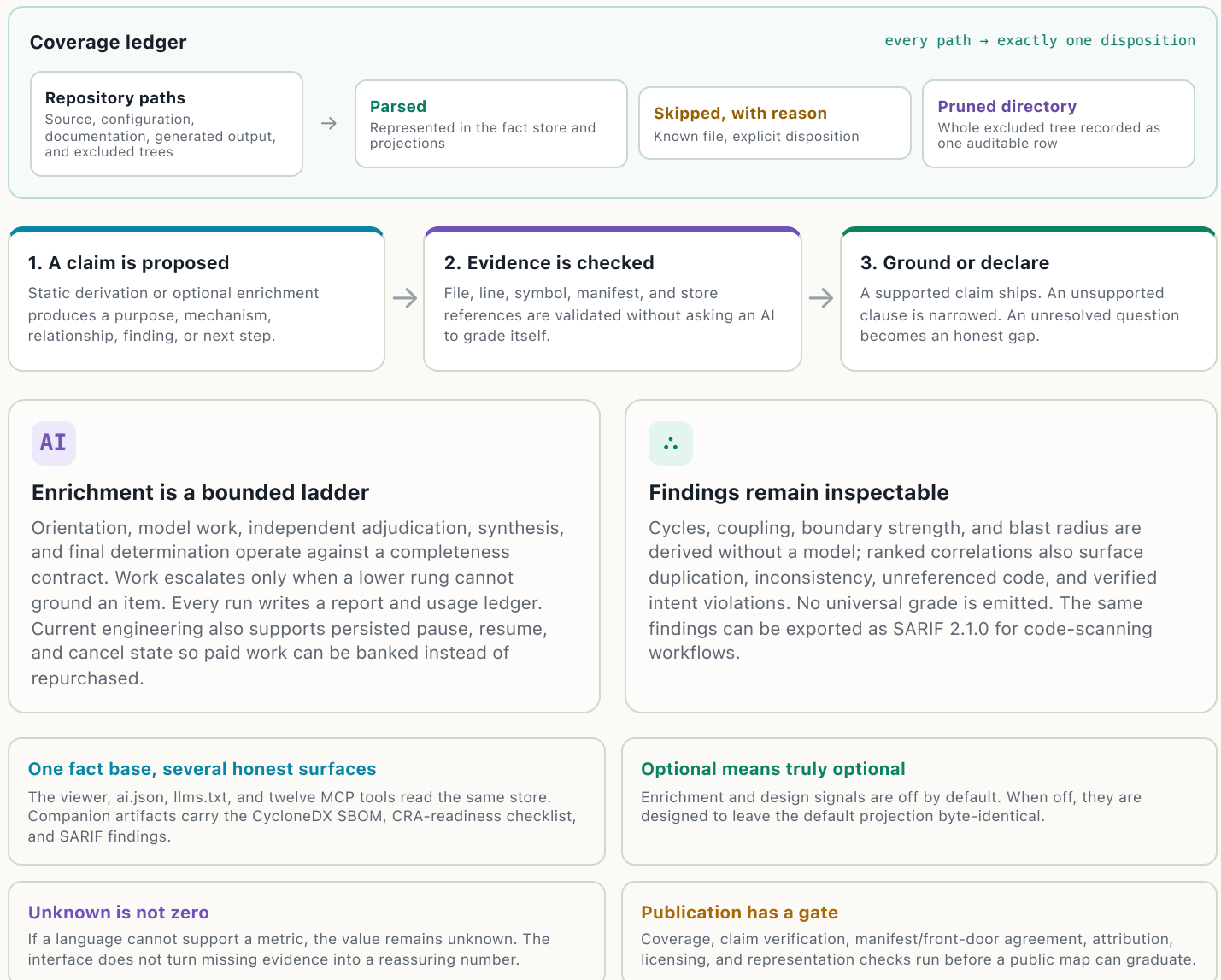
Detail without a monolithic download

Large maps can ship a compact manifest and fetch per-component details only when opened. Search indexes can arrive in shards while the already-loaded portion remains usable.

The map must say what it knows, how it knows, and where it does not know.

Completeness and evidence are treated as data contracts and release gates, not as tone-of-voice promises.

A plausible but quietly wrong map is more dangerous than an obviously incomplete one. The implementation is therefore organized around traceability, cross-surface agreement, and explicit gaps.

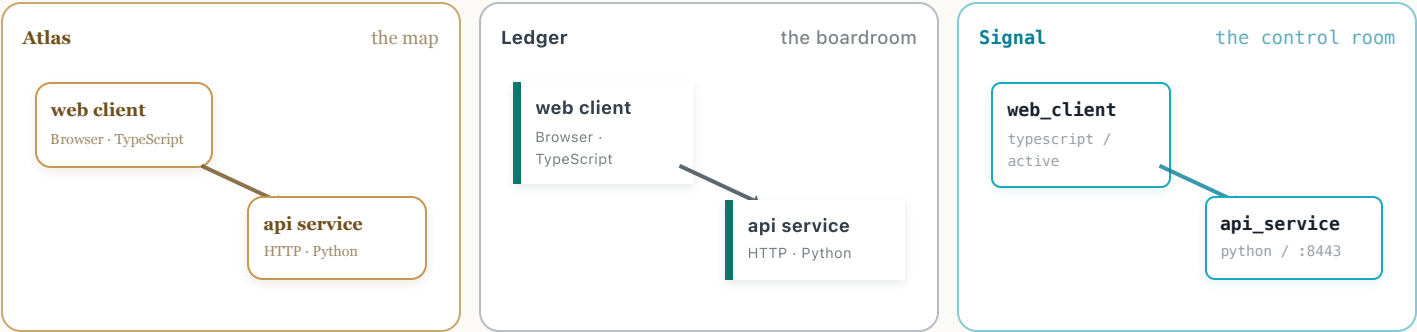


Why the machine front door matters. An agent can ask for an overview, search, a component or symbol, references, impact, coverage, rules, findings, design metrics, or blast radius without rereading the raw repository into context. Results are bounded, evidence-marked, and explicit when more data exists beyond a response limit.

One map can serve different rooms without changing its facts.

The visual system is moving toward a strict separation between the map's meaning and its dress, including organization-specific presentation.

A software map may be read by an engineer at a workstation, an executive in a review, an operator in a control room, or an AI agent with no screen at all. The underlying evidence should remain stable while each surface becomes appropriate to its reader.



The live syscorpus.com homepage demonstrates Atlas, Ledger, and Signal by applying three CSS treatments to one unchanged block of specimen markup. Its numbers are explicitly illustrative. The separation itself is the proof.

Implemented now

The viewer already has dark and light presentation, responsive desktop and mobile layouts, role-specific component frames, and data-driven lenses. The public site carries the working three-style CSS demonstration.

Direction being built toward

Move the production viewer toward the same separation: interchangeable visual skins over one stable semantic model, eventually including customer or organization branding without forking map behavior or evidence.

MODE	WHAT IS DELIVERED
Local / static	A browser-ready bundle that can be opened locally or hosted as ordinary static files.
Single file	One architecture.json for smaller systems and simple distribution.
Split output	A small manifest plus on-demand detail and search shards for larger systems.
CI / scheduled	Automation can ingest changes and publish an updated map after merges, on pushes, nightly, or at another project-appropriate cadence.
Live monitoring	Optional status overlays, version history, activity, and adaptive polling through static hosting or a Cloudflare backend.

- N Node entry point**

A one-command CLI is implemented, along with an initialization wizard and GitHub Action integration. Public registry publication is prepared, not yet completed.
- M Multi-repository systems**

Local paths and remote Git repositories can be composed into one solution with explicit cross-repository relationships.
- C Continuous maintenance**

The persistent store and content hashes already avoid re-parsing unchanged files. Dependency and blast-radius propagation define the fuller v2 treatment still being completed.
- A Portable evidence artifacts**

Eligible projections emit sbom.json and cra-readiness.json; SARIF export is available alongside ai.json and llms.txt.

The skin is not the truth layer. A skin may change typography, density, framing, color, and emphasis. It must not change component identity, evidence, coverage, relationships, or what a lens is allowed to claim.

Substantial machinery exists. Public proof is the next threshold.

Status below is intentionally dated. It distinguishes what exists in the repository from what is under validation and what remains direction.

AREA	STATUS	FACTUAL STATE ON 27 AUG 2026
Core and updates	BUILT FOUNDATION	The v2 fact store, content hashing, coverage ledger, multi-repository composition, parser families, and single/split projections are built. Changed files update without re-parsing unchanged files. Full dependency and blast-radius propagation is the explicit next v2 treatment.
Interactive viewer	BUILT	Hierarchical graph, eight gated lenses, search, source deep links, code previews, annotations, tours, activity and ownership signals, mobile behavior, live status, and administration surfaces exist.
Machine access	BUILT	ai.json, llms.txt, and a zero-dependency MCP server with twelve read-only, evidence-oriented tools are implemented and checked against the same fact store.
Supply chain and SBOM	BUILT	Eight manifest ecosystems feed an evidence-linked supply-chain view with resolved versions, direct/transitive scope, pin state, SDK targets, vendored code, fixture separation, and parse warnings. Each dependency-bearing repository projection emits CycloneDX 1.5 sbom.json.
Security and vulnerabilities	GO-LIVE REQUIREMENT	Useful inputs already exist: network schemes and edges, authentication patterns, environment variables, secret-shaped inventory, rules, dependency pinning, and CRA hygiene checks. A unified security view and SBOM package/version matching against maintained vulnerability feeds, with CVEs surfaced as evidence, are required before go-live and are not claimed as complete today.
Evidence-bound enrichment	VALIDATING	The five-phase engine, completeness contract, adjudication, usage ledger, run reports, quality audit, and pause/resume controls are built. Current work is validating final run analysis and full real-subject behavior.
Public maintained corpus	NEXT THRESHOLD	The harness, private preview, attribution and license review, comprehension instrument, and graduation gates exist. Major open-source maps will publish one at a time, stamped with source and commit and refreshed on a project-appropriate schedule. No subject is announced before its map is ready.

Where this stands, honestly.

SysCorpus is early and independent. The engine, viewer, SBOM and supply-chain surface, quality gates, and comprehension instruments represent substantial completed work. No public subject map has graduated. The unified security view and CVE/advisory correlation are explicit go-live work, not present-tense claims. Engineering stops when quality, completion, or evidence does not hold, repairs the system, and reruns bounded canaries.

Built now • dependency truth Inventory, versions, pin state, scope, source evidence, targets, vendored code, and CycloneDX output.	Required for go-live • CVE correlation Match SBOM packages and versions against maintained advisory feeds and keep those results current.	Permanent boundary • no false verdict A package-to-CVE match is evidence. Reachability, exploitability, and remediation status require their own support.
--	---	---

Near-term proof

- Complete the independent review and bounded enrichment validation now in progress.
- Finish the unified security surface and maintained SBOM-to-advisory/CVE correlation required for go-live.
- Graduate the first real public map through coverage, security, quality, attribution, license, and comprehension gates.

Longer product direction

- Grow a maintained corpus of recognizable systems and show architectural, dependency, and vulnerability changes over time.
- Refresh enrichment and security intelligence while detecting stale narrative, tour, and advisory evidence.
- Carry the visual wardrobe into production, including branded skins over an unchanged semantic map.